ActInf MathStream 013.1 ~ Alexis Toumi: "DisCoPy: Monoidal Categories for Active Inference"

Source: YouTube **Playlist**: MathStreams **Duration**: 1:06:48 | **Uploaded**: Unknown **Transcript method**: auto_caption

---

Hello, welcome everyone. It is May 20th, 2025. We're in active math stream number 13.1 with Alexi Tumi discussing disco by monoidal categories for active inference. This should be a very exciting and useful presentation and stream. So, thank you very much for joining and looking forward to learning more. Yeah, thanks a lot. Um, so thanks a lot for the invite. Um, yeah, I guess um, as I was saying just a couple of minutes ago, um, I'm very much looking forward to this talk and it's a bit of a challenge for me, I guess, in a sense that I'm no expert in active inference. So, I'll I'll do my best at explaining what I think active inference is about. And uh I also try to uh yeah convince the audience here that um disco uh this Python library uh for uh string diagrams is um promising way to to experiment with active inference. So I guess um most of the the references I I'll be using for for today would be this uh main disco pi paper. Um, you can also check out the the documentation on on GitHub. It's quite extensive. We've got plenty of examples and notebooks to play around with. Um, and then I'll give a a small glimpse of what the disco software is used for. So, this idea of QNLP, quantum natural language processing, which is what motivated the the library in the first place, what I wrote my PhD thesis about. um and um I guess I'll give some introduction to category theory uh which is the sort of mathematical foundations on which uh this copy is built and then I'll be using mostly this um recent uh paper from uh to Kleiner and SM on active inference in string diagrams uh where essentially they um use the same mathematics that I was using for quantum NLP and uh apply that to uh to active inference and in a sense uh that's where I learned all I know about active inference. So I'll be giving a a summary of the sort of theoretical background. Um and then I'll be mostly showing some screenshots from that paper to try and show how that theory applies to active inference in particular and then then we can discuss u how disco could could be used um to implement active inference experiments. So yeah let's uh let's get going. Um so I guess the main um theoretical concept in disco is a box. So, like we we we draw these boxes a lot. And I guess uh I um I don't roast coffee every morning, but I I

grind coffee. I make I brew coffee every morning. And I I guess that's a something that you're going to understand in terms of a black box where you you you put a bunch of uh inputs like green coffee, beans, electricity, roast level, start signal. Uh it could be physical, it could be information. These boxes are sort of abstract um conceptual operations that just have uh a name, a designated list of inputs and outputs uh and a designated list of um so yeah a designated list of inputs and a designated list of outputs. So you've got two sets uh the objects so the things are on the wires uh and um boxes um um have have names too and then you have these two maps from boxes to lists of wires. Um so that gives you all the processes that you can talk about in your in your theory. Um and then you can translate one theory into another using these morphisms of signatures. So uh first if you have a box from X to Y, you draw it with an arrow in this way. Um and then if you have a morphism of signature, what it means is you can translate wires of one into lists of wires in the second. Um and then you can translate a box in one into a box in the appropriate thing in the other. So if you have um so this should be sigma prime. I guess it's a typo but you translate boxes in a way that respects their inputs and outputs. Um and so that's very useful because if you have different uh kinds of of processes and you want to translate between them um that concept of morphism of signature is going to be the the key one. Um and so I guess um with that you can define um the set of diagrams. So that's going to be the main um the main uh definition for this thought. Um and you start with a signature and then you define diagrams by induction. Uh where every box is a diagram. Uh the identity is a diagram and you draw it with just a bunch of parallel wires. Um the composition of a diagram from X to Y and Y to Z is also a diagram. So you can if you have two uh diagrams, you can plug them one after the other. And then uh the tensor of two diagrams is also a diagram. So if you have f and frime, you can put them side by side and you get something that goes from xxx prime to y prime and that's it. So you can um put um diagrams in sequence and in parallel and you can also do nothing. So this identity uh diagram um and then I guess the subtlety that we'll use here is that um well I guess first you need to put some aums on these. Uh so I guess you want tensor and composition to be associative. Um you want them to be unit. So if you compose with the identity, it doesn't do anything. If you tensor with the empty diagram, so the identity on nothing on the empty list, um that's also doing nothing. And then you want just these two equations that tell you that you can uh switch identity and tensor. So ID of tensor or tensor of ID is ID of tensor. Um, and similarly for the composition, the tensor of two compositions is going to be the composition of the tensors. And we'll see what that looks like in terms of diagrams in a second. But as I was saying, there's this

sort of uh nicity that you get is that if you take a signature uh and you look at all the diagrams uh that you generate from it, that makes a new signature and you can use that as a signature where essentially now the boxes have diagrams inside them. And so that's a way of formalizing an idea of hierarchical diagrams where essentially a sub diagram will contain or a sub diagram is a box that contains a diagram inside it. Um so we'll use a lot of these hierarchical diagrams where we'll see the the boxes serve u to encode normalization. Um, and so yeah, to get back to this main equation, I guess that's the only equation you need to remember. And you don't actually need to remember it because essentially you got it built into your visual system. So that's the sort of um main argument for why you should um you should use string diagrams is that essentially they encode this very natural way of looking at processes as uh things on the plane where the axioms become um the axioms become um just manipulation of these uh of these diagrams geometrically. So for example this um this axiom of naturality where um you exchange the composition and the tensor this amounts to having either uh the boxes composed uh first sequentially and then in sequence or first in sequence and then sequentially and that gives you the same um the same diagrams in the end. So well the only thing that counts is how things are connected I guess is the the main idea for that axum. And so you can apply that to cooking. Um the ingredients are going to be your wires. Um the steps the cooking steps are going to be your boxes. And now anything that you can compose with these cooking steps will be a recipe. It takes a bunch of uh inputs like eggs and gives you a bunch of outputs like whites and yolks, right? And yeah, I guess um this this aum of exchanging tensor and composition essentially it amounts to formalizing what we mean by independence. So if we have two boxes that are not connected like cracking uh here and cracking here, we can perform them in any order. Uh if independent, it doesn't matter how we decompose this tensor of composition. Um so yeah I guess um let's look at what this diagram looks like in Python. Um so that's the main feature of disco is you get this very nice syntax for expressing any such string diagram as uh essentially as a Python function. So you you start by declaring some types for the ingredients of your recipe like egg, white and yolk. You have a box for cracking an egg and giving whites and yolk. You have these parameterized boxes. So merge, you can merge anything. So any X and that takes two X's and gives you an X. And now to define the diagram on the right, you'll just define a Python function. Declare its inputs and its outputs. And now if you take X and Y for the two eggs, you crack them, you get AB is a white yolk pair, CD is a white yolk pair. So you can merge the whites together, merge the yolks together and return. So you get your diagram and you can draw it. So this output on the

right is actually what disco gives you as a as a drawing. And you can also write it in terms of purely uh pure operations of tensor and composition. So this uh add symbol is how you do the the tensor the parallel composition and this these chevrons do the sequential composition. And here you notice that we need this special box for the swap, right? Like uh at first with just tensor and composition, you can only do planer uh processes. So here you need this special swap box that takes yoke and white and gives you white and yoke. Um so yeah, I guess that's the sort of simplest example of how to use this copy is you can define a bunch of types. um define a composite model uh composite um diagram out of these uh basic uh steps, basic boxes and and types and then you can draw the result. I guess drawing the result would be the first step. Uh and in a second we'll look at um what else you can do with diagrams. So a more interesting example I guess in our context will be um a generative model as a diagram. So on on the left you have the sort of basic example of a probabilistic model where you have random variables numbered x1 to to x5. Um and then you have these arrows. Um I guess um here some arrows are designated as sort of input arrows. Um here on on the left uh x2 and x3 are sort of variables that I guess you get to control or you you can choose some um some distributions over these. Um then you have some hidden variables like this x1, x2 and x4 and you have um observed variables right the x5 and x3 would be the things that you can actually observe in the end. And then sort of implicitly in every such um probabilistic model graphical model you would have a conditional probability distribution for every variable given the variables that point to it. Right? And so in the world of string diagrams, these conditional probability distributions become the the important thing in a sense like the I guess here the labels you could almost forget about these labels X1 to X5. They help to make the correspondence here. But the actual important operations are these conditional distributions. So X1 well you need a distribution on it. It's this uh latent variable. So that's going to be your A. It's going to be a prior on on X1. And then you you need a distribution for B that gives you X4 based on X1 X2. You need a distribution C that takes uh gives X5 given X4, X2, X3 and so on. And then here you also need this important operation that does copying, right? So here the variable X2 will be used um in the in the conditional distribution for X4 and for X5. So you'll feed it to B and you'll feed it to C. So you need to copy it, right? You need to broadcast this um uh whatever input you feed in here, right? Um and so if you take um a distribution that's uh non-trivial, well, this will create some in this will create some u correlation between those distributions, right? Um and so at the end, what you get are the observed variables which are these outputs of your of your diagram. And so you get this mechanical translation where every

um every probabilistic model can be faithfully encoded as such a string diagram. Um the the wires are the random variables. Um I guess the subtlety here is like the wires um up to this copying right. So here the variable x2 appears uh twice. So you need to split the wire in some some formal sense. Um the boxes are conditional probability distributions that define your model. Um and the diagrams are the generative models themselves. Right? So here's what the same code would look like in Disco Pi. Again, you're going to instantiate a bunch of types for X1 to X5. And then you're going to have these boxes ABC. And you can define your model as a Python function that takes a bunch of variables and just does the usual thing. And I guess the magic of of these Python decorators is that your function here is not going to be interpreted as an actual function. It's going to be executed sort of symbolically. And these variables X2 and X3 are going to be turned into the input wire of your diagram. And then these x5 x x3 will be the output wires of your diagram. And so in a sense what disco pi gives you is a sort of domain specific language where you can write any such uh generative model and you will get a data structure for it. So I guess today I won't have much time to get into the the data structure underlying these but I I guess there's sort of uh two main ones two main ways of understanding these diagrams is the first one is as a sort of sequence of layers where each uh layer of your diagram is like for example this first layer you've got a box A in parallel with copying in parallel with another copying the second layer would be B in parallel with a bunch of identities and the last layer code BC. So this sort of list-like representation is very convenient for drawing. And I guess if we go back to our cooking example, this is why you get to see these sort of stacking of boxes is essentially it's more convenient to have one box at each layer. So you just get a list of these guys, a list of operations that you perform. Um and then there's sort of the graph-like representation which is the more natural one when you think of a graphical model. They're called graphical because it's a graph, right? So I guess discoy gives you this ability of going back and forth between graph like and sort of list like and the listike operation is also how you define um the evaluation of these diagrams. So we'll get to see that in in a minute. These diagrams you can draw them you can store them as combinatorial representations of of graphs. Um and I guess the main point is you can evaluate them and you can do experiments with them. So let's get to that. The I guess the the theory we need behind this is u is that of strict monoidal categories. So I'll just say category for short um because I guess they'll all be strict and monoid uh in this talk. And essentially um once you know what a signature is. So this set of um of of wires and boxes with domain co- domain you can define a category as a signature with a way of evaluating diagrams essentially. So a way of

taking a composition of boxes and wires to give a single box. So first you need a way of interpreting identity uh for each object. you need a way of of interpreting uh composition and you need a way sequential and parallel composition right so now essentially a category will be a concrete way of interpreting these diagrams um and I guess um the the key sort of structural theorem for well okay let's first look at some uh some examples um so I guess the main example we've seen so far of a category is the category of wires and diagrams where the objects are these these wires the lists of inputs or outputs and the diagrams are this composition of boxes that you can represent combinatorally um as graphs. Um and then um the I guess the most natural interpretation of these um if you come from some sort of mathematical background would be to think of sets and functions right so every box will be a function of its input wires into its output wires. So you think of cartisian product or you get a multiple inputs and you get multiple outputs. Um I guess sets in and relations are super interesting too. Uh instead of um so I I guess in that case you can you can do a lot fancier diagrams. Um if you think of relation so you don't have inputs and outputs anymore but you can essentially think of them going both ways. Um then I guess yeah um the main example of these monoidal categories uh was to do linear algebra to sort of formalize u things that physicists were already doing uh drawing diagrams linear maps as diagrams. So I guess that's the main example. Um and in our case what we're interested in uh will be more something like the category of measurable spaces and stoastic maps. So we'll think of each wire as a measurable space with the product space being the tensor the the operation that puts wires side by side and then um a box will be um essentially a stoastic map that takes a product space and gives you a new product space. Um and so I guess with this interpretation in mind um essentially a graphical model a pro generative model is precisely a diagram with an interpretation in stock. So I guess yeah um the then the main way to formulate this idea is essentially to define what's called a funtor um is going to be a morphism of categories. So something that translates a signature into another and also preserves identity composition and tensor. Um and then you have this sort of main theorem that tells you that diagrams are the free monoidal category. So what this means uh intuitively is that if you want to define a funtor from the category of diagrams, the only thing you need to do is to define an interpretation for each wire and an interpretation for each box. So on the left hand side you've got all the funtors from C sigma to D. And here you've got all these sort of much smaller objects where um essentially you just need uh measurable space for each of the wires and uh a stochastic map for each of the boxes um and or conditional probability um and now you get an interpretation

for each composition of these. Um, so that's sort of the the structural theorem, the sort of foundational theorem that justifies why are string diagrams the right data structure when you're um looking at composition in sequence and in parallel. Um, any composition, any such composition is a diagram. Um and so I guess with that theorem in mind like on the right hand side you've got something that you can represent as a small data structure that uh that that is uh you just need uh the representation for a stoastic map for each of the boxes in your diagram. And on the left hand side you've got the sort of algorithm that's going to take these diagrams and compute their their meaning by plugging these um distributions together and essentially performing inference. So on on the right hand side you've got you've got something um that is sort of data like on the left hand side you you've got the algorithm that's gonna compute that data to give you um inference algorithms. Um and I guess yeah I was um promising some some short um detour into uh quantum uh natural language processing. So I guess here it's just a small code snippet to um show I guess how concise you can make um a particular model if you write it as a functor. Um so here uh we've got boxes are worlds. So we've got a a box for Alice, a box for love, and a box for Bob. We're going to compose them together uh according to their grammatical structure. So here the the the word Alice is connected to love via this cup and the word bub is connected to love via that cup. And overall you get a sentence. Um so that's the sort of natural language side of things. And then you get the quantum side of things by saying look every noun I'm just going to interpret it as a cubit. So as a quantum system that I can control um the noun alice is going to be one state of the cubit the state zero. The word bub is going to be the state one. And now the world love is going to be where interesting stuff happens where I'm going to entangle or correlate my inputs and my outputs my subject and my object. And so here you represent it as a quantum circuit that's going to be some some hard gate and some control gate. So essentially a bunch of quantum gates represented as boxes that you compose in sequence and in parallel. So I won't get into much quantum theory here, but I guess the the main punch line is that you can replace quantum by um probabilistic um circuits and the same theory applies in a sense that um you get this compositional way of interpreting well natural language but interpreting any sort of compositional system in general. So here you take your sentence. You apply the funtor to it. You will get uh a circuit a quantum circuit and then essentially this eval is going to be a second step of the funtor that's going to take your quantum circuit and turn it into um a bunch of matrices. So it's going to turn it into vector spaces and linear maps and compute the meaning of that circuit, compute the meaning of that sentence. Right? So I guess that's one

QNLP experiment in um in a few lines of Python. And I guess yeah the argument for this talk would be that we can replace quantum with uh with active inference here and and get a very similar recipe for how to build uh larger um active inference systems from smaller ones in their interaction. Um so yeah I guess that was my thesis three years of PhD thesis in one slide. Uh so let's uh let's move on. Um the yeah I guess I'll try and give some of the sort of necessary theory to um to look at active inference formally uh but give it all in terms of diagrams rather than complicated formula. So now that you've seen a bunch of diagrams, uh you can you you've noticed that you if you only have tensor in parallel and in sequence, you can you can only do planer diagrams. If you want um if you want arbitrary diagrams, you need this special swap. Um and the swap is going to just satisfy these four aums you see here. If you swap with nothing, it doesn't do anything. If you swap and swap back, it doesn't do anything. If you swap F and G, it's the same as doing G and F and then swap. And finally, you can decompose a complex swap in terms of basic swaps. So that's all you need to get a symmetric monoidal category. Um, and essentially in in a symmetric category, you don't care about the order of wires because you can permute them. So in the case of probability theory, you don't care about the order of your probabilistic model. you can permute the variables. Um what matters is what's connected to what via boxes not the order of the inputs and the outputs. Um and um yeah I guess in natural language you do care about the order of words. So in that case you're not symmetric. Um but um another kind of category once you have symmetric is um is the idea of a compact category and that's where you have the ability of bending wires. So I guess in a normal string diagram the wires are going to flow from the inputs to the outputs. Um in a compact diagram you can flow backward and um in the first paper applying category theory to be basian inference that was the technical apparatus to formalize the idea of um Beijian inversing. So I guess with these cups and caps you can take um a joint distribution on a * b and bend one of the legs to get a stochastic map from a to b. And similarly if you have um if you have a stoastic map you can bend the output and get u a joint distribution. And now uh if you have a stoastic map from A to B and you have a prior on A um so I guess that's the the little trick that they had to hide is this little box uh this little black box on on the cup here. Uh then you can invert um you can perform Beijian inversion and you will get um a stoastic map from B to A. Um and I guess um yeah what there are many ways of formalizing this uh caveat of you need a prior if you want to be able to do uh Beijian inversion um and yeah I guess I won't get too much into the the literature of how you you formalize uh categories of uh stoastic processes are a very big topic but in that context having a compact category means

essentially every object every uh wire comes with a prior um and now when you invert you use that prior um so yeah I guess in quantum it's got a very different interpretation where these wires are going to represent bell states entangled states and entangled measurements and then this equation amounts to teleportation in the context of basian probability uh this equation is just the basian inversion of uh a trivial probabilistic channel um is itself. So I guess not very interesting in quantum it's like wow we can teleport things in in Beijan. Uh it's just the inverse of the identity is the identity. Um so that's yeah that's a compact category. Um you can bend things around. So if you have symmetric plus compact you can truly talk about a graph structure for your diagrams because you can connect any input to any output uh in a graph-like way. Um so now I guess an important ingredient um we've seen for um defining a generative model is the ability to use the same variable in multiple places. Um so that's formalized in terms of these common monoids. So you have an operation that splits a wire and it's um co-commutative. So if you split and then you swap it's the same thing as just uh splitting. Um it's got a unit which is discarding. So if you split and then if you if you copy something and then you discard or if you split and then you terminate a wire you haven't done anything and it's coassociative. So it doesn't matter if if you copy the left hand copy or the right hand copy you get three identical copies. Um and yeah I guess the main two axioms for being a cartisian category are these two. So the the second one I guess um you can call normalization. So essentially in terms of stoastic maps what this means is that if you um apply a stoastic map and then take the weight of the result or take the mass of the result um for any distribution coming in here it's the same as the mass of the result uh as the mass of the input. So essentially if you have a normal it takes a normalized um it takes a normalized uh distribution to a normalized distribution. Um and so on the left hand side is more controversial is not going to be true in the category of stoastic maps. Uh this says that f is deterministic. So if you perform f and then copy the result is the same as copying the inputs and performing f twice. And I I guess if you take the special case where f has no input, this means that essentially um f can be copied means that um it's deterministic in a sense that performing f if f was a toss coin, right? Um you would take f throw throw a coin and then copy the result um you would get a different result as if you cop you you you toss two coins, right? So um the second one um is going to hold in terms of Beijian probability theory if you have normalized processes and it's this idea of essentially you can also formalize it as causality. So if you discard the output it cannot influence the input. Um and um yeah the first one is essentially what brings you back to the world of deterministic processes. So it's a good way of characterizing which stoastic map is in

fact a deterministic one. And so if you have um if you have these two axioms you're called a cartisian category. If you don't have them you're called a copy discard category. And so I guess um in our context we're really interested in copy discard categories where these two are different processes. Um and yeah, I guess what what can you prove at this abstract level of of generality um is a a really nice no cloning theorem where you don't need to assume any complex numbers or any quantum stuff and in fact it holds as well in the context of probability theory. Um and it's I I think it's called the no broadcasting theorem in that in that case. Um but the the idea is that you cannot have both cups and capsu uh and copying. So here I guess you assume that you have cups and then because you have copying then the the copy of a cup is also going to is going to be two cups right. Um but you can also uh you can also do some shenanigans where you you bend the wires because copying is co-commutative. So on one side you're going to you're going to you're going to bend the wire. You're going to swap the wires. And now you've essentially proved that two cups like this side by side are actually equal to two cups nested inside out. Um you bend all the legs and you get that a swap is equal to the identity which is basically um yeah um it means you've done something wrong, right? like if you can't even distinguish swapping stuff from not doing anything, your category is uh very much trivial. So I guess that's the main the no cloning theorem is that if you're both compact, so you have cups and caps and copying now uh you have to be a trivial category in the sense that everything is a essentially a scalar multiple of the identity. Um so um that's not the case in probability theory. So probability theory also has this no cloning feature and that's essentially well you can't copy uh you you can't copy something if it's a non-trivial distribution you can't copy uh a correlated u state right um the same way you can't copy an entangled state in in quantum um so yeah um I guess um the last ingredient we need in order to talk about active inference is the idea they have normalization um and it in the context of string diagrams that's going to be represented as I was saying by hierarchical diagrams where you have a sub diagram that's uh inducted lines will just represent the normalization of it with some caveat if there's no support for the um for the particular state. So um here you have um a stoastic map from X to Y and you put it inside of a dotted line and you get its um its normalization. So I guess the guy inside is not really a stoastic map anymore. It's more of a just any measurable function. Um not necessarily normalized and the thing outside becomes a proper stoastic map. Um and that means for any um measurable function you can decompose it in terms of uh something that's uh normalized and then something that you discount. So these three lines are the the discounting operation uh that we drew as a as a white dot before. But um so essentially

here that equation tells you that every measurable function is going to be a normalized one up to some renormalization. So I guess it's sort of intuitive and also normalization behaves nicely with respect to parallel composition. So if you have f and gum and then par you normalize them you get the normalization of f thanks to the normalization of g it behaves well with respect to copying uh a particular um variable of your generative model. Um but the thing it doesn't behave well with is the composition. So it's uh not true that the normalization of composition is the composition of normalization and that's um essentially what gives you the difference between a Jeffrey update and a pearl update in the in the context of of Beijian inference. Um so on the left hand side you've got a Jeffrey update where you first normalize and then pre-ompose with the prior. on the right hand side you've got your pearl update where you do um first the pre-omposition with the prior and then you normalize and so I guess um here on the left if you want to obtain something normalized over row you will need to renormalize so like you get two normalizations on the right hand side you only get one and that's sort of the difference that's the the fact that normalization is not a funtor in a sense it doesn't commute with the composition Um so um then another kind of bubble we'll need is uh taking the logarithm of a function. Um so if we have a function from x to the reals positive reals um we can take its logarithm this function is just going to be uh if we if we think of the category of measurable functions well it's um it's um the empty diagram is going to correspond to the real numbers to the one-dimensional space and so um such a function will just be a box an effect which takes uh something and gives you just reormalization factor, right? Um and now if you have such an effect, you can put it into a green box and you'll get the uh the negative log of it. Um so that's sort of a mere um sort of notational gadget. Um and essentially yeah it only becomes interesting if you start using it um inside a more complex diagram where this um this green box will not in will not commute past the composition and will give some interesting hierarchical structure to your to your diagram. And I guess here I'm just copying some caveats from um from the the to um to Ital paper. Um, and yeah, I guess they they give some some pointers to some interesting category theory that could be formalized to um to to explore these log boxes further. But um in our context where we need them to is essentially yeah we just want the the nice properties of logarithms. So if uh this copying operator acts like um essentially a product uh then we want that the log of a product is the sum of the logs. So you get this sort of nice um nice operation um nice um yeah homorphism property of the log um the discarding operator it acts like the unit of this um multiplication. So the log of the units will be uh zero. Um and then you get this nice product rule um this sort of lightnit rule for the

logarithm of of a tensor product in that sense will be the sum of two tensor products. So that's also the kind of equation you'll find in a sort of automatic differentiation uh framework where the gradient of d tensor e is going to be the gradient of d plus the gradient of e. Um and finally if x is deterministic so that's what they call sharp states. Um then you it also commutes past the the log. Um so you get these nice property of the the logarithm which allows to take a diagram with these green boxes and sort of massage them into uh a sum of diagrams. Um and finally you can define uh this notion of open vibrational free energy where instead of looking at the free energy of a closed model uh so something that wouldn't have an input I here um so just a joint distribution over states uh internal states and observed variables you have this idea of an open uh model so you have some of the inputs that you can control um and Now you can define the free energy of this model with respect to joint distribution on uh the inputs with respect to the and the internal state and uh a prior on the observed variables and uh here's the formula. So I guess that was sort of the the main uh point of the paper was to build all these categorical apparatus to get to the point where you can just um draw the definition of free energy as diagram. Um and then the main theorem uh being that essentially uh this gives you a compositional um definition in a sense that the free energy for uh composition. So here it's the tensor composition. Um the free energy of the tensor will be some composition of the individual thing. So I guess in the case of tensor is the simplest case. uh I won't show the the case for sequential composition where you need to to take um a more uh complicated uh operation to compose the the subsystems. But I guess that's sort of the the main theorem of that um categorical active inference framework is that you get um this idea that the free energy for the whole is composition of the free energy of its parts and essentially yeah mono category theory gives you the sort of um the high level uh layer of abstraction to describe these sorts of compositional phenomenon. Um and yeah I guess with this uh sort of foundational theory um you want to turn into application and so I guess that's what this copay was uh built for. So what we already have um to support this sort of of active inference implementation in terms of diagrams well we we have most of the finite dimensional case uh implemented already in the sense that if your u if your random variable span a categorical distribution is a categorical distribution then you can turn this basian inference problem into a tensor network contraction problem. Um we have a bunch of automatic and diagrammatic differentiation uh algorithms implemented uh with some um compilation uh with the JAX library. So essentially you can take your diagram and compile it into a bunch of GPU code for very fast execution. Um and finally um the sort of more experimental more speculative um part of the library is this idea of

the the geometry of interaction where you formalize birectional processes. So sadly I won't have much time to to get into um that side of things but there's this very interesting idea of a basian lens where you have um these birectional operations that also have some stochastic side to them. So essentially you can talk about uh updates about the the forward part of your process being some inference and the backward part being some update and this geometry of interaction will give you a way of composing these updates together um in a compositional structure of vision inference. So um we already have the sort of purely uh abstract implementation of this geometry of interaction. What's missing is sort of plugging these two things together. So just looking at how if you take finite dimensional as a first step uh and you look at how it updates um you should be able to implement it in a purely compositional way uh with disco and also be able to draw the result in the end um and I guess what's missing would be um a more um continuous um um implementation where um I guess to go beyond finite dimensional uh you need some sort of um yeah more involved implementation based on monads I guess is a very well-developed um theory on the category side um and there's one thing we've started working on which is this idea of delayed interaction where um essentially you can represent discrete time processes so you have a very natural way of talking about time series time um and yeah I guess what the the the main ingredient that's missing to develop this this further would be having experimentalists are ready to to take the challenge to build their experiments in a way that you can compose them with other experiments in the future and build it upon uh a theory of how to compose processes together. So I guess um that was sort of the main uh objective of disco was to sort of lower the entry to uh the cost of entry to understanding string diagram so that any experimentalist from quantum computing or active inference can pick up just enough uh abstract mathematics to turn um turn these experiments into something you can compose together. Um, and yeah, I guess um I'd be happy to to get some some questions from the audience and looking forward to um to more interactions with the the active inference community. Um, thanks for the thanks a lot. Thank you. Great talk. Okay, anyone watching live can write a question in the live chat and I wrote down some questions as well. First though, just to kind of pull back like how did you get into writing Disco Pi? what brought you to wanting to build this toolkit and what made you interested in the area? Yeah, I mean so I guess it started with the the master's thesis in Oxford was um mostly quantum stuff uh with the course that Bob Kirk was teaching there and um I was interested in AI already and NLP in particular at the time and I guess um yeah there was this program of bridging the two together and making quantum NLP uh and so I I got involved in

that and when it came to implementing it, it felt like it would be a shame to implement uh quantum NLP uh as a as a oneshot thing where you build your experiment and throw it away. And I guess um we took it as an excuse to develop something that we thought was much more broadly applicable where you take this um general theory of monoto categories and implement that and use Q&L as a sort of proof of concept that it can do something useful and then hope that other people will start formalizing more stuff as monoid categories. So I guess this uh paper from to Ital came much after the the thesis and um I guess um they took active inference as as um something that you could formalize with diagrams too. So yeah um I I guess QNLP was a bit of an excuse to get into some very cool maths and to develop some some cool software to back it up. And now I'm sort of yeah looking forward to to seeing other ways to to apply this software. Awesome. Okay. I'm going to read and answer one question from the live chat and then I'll ask you a question. So Dival wrote is this under a specific group project at active inference institute like ontology or other project. So just to briefly address that it could be super relevant for ontology. We could use the active inference ontology to extract natural language descriptions of generative models for example represent them in disco. So it could be a tool used in ontology project or it could be spun up as a category theory project as we've done in the past when we for example focused on some of Toby uh Sinclair Smith's work. So everyone's welcome to uh get get involved and reactivate projects as they are excited by. This is definitely something exciting. So, so let me ask you this question. How do we connect the properties of diagrams for example the ability to swap capture causality and the error of time cups and caps and all that with the features of cognitive systems which we're interested in? Like how do we know which kinds of formal constraints are overkill, nonsequittor? How do we really like reconcile and build with the right formal systems when we're dealing with embodied cognitive systems that don't on a first pass have those kinds of constraints? Yeah. Um I think that's a super interesting question and in a sense like not being an expert in active inference I don't feel entitled to to really say what are the properties that you guys are interested in but I guess as a sort of parallel to other ways diagrams have been applied in in particularly in quantum I felt like one interesting way to look at at diagrams is to to look at them in terms of comtorial structures that have some um some structural properties that you can exploit to turn the turn the computational problems associated with them into tractable problems. So I guess the maybe as a as a bet I would say that characterizing which models are efficiently simulable simulatable or not is sort of a key question that you ask yourself when you you you get to build a model of of um cognition in general in a sense that if you cannot

simulate it, if you cannot um um build an efficient implementation for it, then it's likely that the brain isn't going be working that way in a sense the brain is efficiently solving uh this cognition problem. So behind it there must be some sort of um tractable algorithm and I guess there's a way of characterizing which um problems will be tractable by the shape of the diagrams associated with them. So one particular um way this this happens is uh when you look at tensor networks um uh if you can bound the the width of your diagram and make it look like it's uh almost a tree and you have this idea of bounded tree width where um you can show that your uh your network has some sort of hierarchical structure to it where it's shaped in terms of a main route that splits into branches. Even if these branches are thick in a sense, it's not necessarily a tree, then this bounded uh tree width will give you a bound on how hard it will be to compute this tensor network. So in a sense um from the the sort of graph structure of your diagram uh from the fact that this graph has a nice decomposition you can u exploit that fact and turn it into an efficient algorithm um to solve uh inference on that model to um contract the tensor network in the case of physicists. But um I think this general principle uh applies in a very broad range of cases. Um and there's some like very deep um intersections with complexity theory where um I guess this P and NP question is all about like how do you separate between things that you can solve efficiently or not, right? Um so uh diagrams give you an interesting way of looking at this P versus NP question where the yeah I guess you can try and tell apart these two classes by looking at the shape of diagrams that represent these problems. Awesome. Okay. So, what makes the variational free energy open or not open? And then some more VF related questions. How how did you connect that to the log properties since of course the free energy is used as a bound on log probabilities as surprise. And then what does it mean that the VFE is compositional since from a numerical perspective the compositionality is kind of trivial? Like of course I could calculate the free energy for two different situations and just add those numbers together. So what do we really get with the semantics of composition that go beyond just calculating two numbers and then adding those two numbers? Yeah. So um I guess to go back to to that definition of uh open free energy um I guess the yeah the main difference here compared to the the standard definition is this extra wire I right so you I guess um the the idea is that if this wire uh is empty uh if there's no particular input you get back to the usual definition um and it um It's the idea of instead of trying to compute the free energy of a big monolithic system that represent your the overall thing you want to study. Um you first decompose that into a composition of uh different things being independent and being composed in sequence. So that's where you get the the

correlation and this decomposition of your problem will give you a formula to compute the the the composition of the the corresponding free energies. And I guess um this example wasn't necessarily the the best one to pick. So here it's the the fact that it's compositional with respect to independent uh uh parallel composition. Uh where I guess yeah it's a bit of a trivial property of free energy that um the free energy of the tensor of two things will be the sum of their free energies. Um and I guess um I yeah um I should have included the more interesting case where you have a sequential composition of things and where this um sequential composition gives you a non-trivial way of decomposing the formula for your free energy. So I guess that would be the the place where you can get some some traction in a particular experiment where essentially um you can yeah I guess the same way that an automatic differentiation uh framework would um would use the structure of your program to compute the structure of its derivative. Well, here you could have um yeah, I guess automatic free energy idea of the structure of your um the structure of your model informs the structure of the formula for its free energy and potentially implying some um some algorithmic speed up. So I guess here obviously there isn't any speed up to be found in this particular um in this particular um instance of compositionality and um yeah I guess I I'm yeah I don't feel qualified enough to uh advance any conjecture on how the the non-trivial case would uh give you algorithmic speedups but I guess that's the the sort of of promise and there's there's the idea of a speed up there's also the idea of having software ware that's more modular. So having software that um for one experiment that you can potentially reuse to compose with a later experiment. Um yeah, like where where that kind of lands for me is we can use these nested box structures to reflect the articulations of different causal and spatial and temporal systems in sequence and in parallel and also proposed cognitive mechanisms. So not necessarily related to the material connections on substrate but related to different kinds of interactions amongst like attention, memory, all these kinds of cognitive features. So, first there's the benefit of the interpretability, legibility, modularity, reuse, angle, and then getting to that speed up. If it's possible to have strong decomposition composition type rules, large boxes with a lot of stuff in them, whether that's 100 nestmates in a multi- aent simulation or 100 different cognitive systems in a complex single agent simulation, those can be brought down to their most atomic computational rules and possibly certain kinds of calculations. will have certain methods that would be like the most effective for this kind of distribution or for this size. So that gives a unified way to look at the topology of information flows and calculations in a notational disco pi framework rather than those connections being kind of semi-implicit in the control flow of a

Python script or maybe scattered across different files with different functions calling each other, not necessarily in a unified way. For example, a common approach is like to have an agent class and then have them be communicating back and forth with an environment. And that could still be really well structured, but it leads to kind of a fundamental division between what's happening within the agent and within the environment. And that's understandable and a useful design pattern. However, the stronger direction to kind of steer towards would be unifying systems for notation possibly like disco pi that could get us like the best of both worlds with well ststructured interfaces and unification across really different representations. Yeah, I think one also one important aspect um is is this idea of separation of concern between syntax and semantics where uh you want the ability to describe your models in a syntactic way that will be reused in different cases with different semantics in mind, different ways of computing inference on the same diagram, right? And so with this idea of uh separating you have the diagrams on one side that represent the um description of your model itself and then you have the funtors that describe what the computation you do on that model. And I think this separation of concern means that well you can compose the syntax together and compose these descriptions to form a larger description. Um and then that way of building your model will inform the computation side of things. And um I guess yeah speedups are sort of the one of the theoretical results you could or experimental results you could get out of this. And I guess I mean other ways of thinking about it. I think when you're talking about topology, it really um it really feels like there there's some connection to be made there in a sense that a lot of what people do with these diagrams is try and embed them onto some topological space where your computation doesn't live in the abstract anymore and is actually embedded in a particular space. And I guess for physicists this will be like embedded into spaceime you know so you will have like these relativistic quantum uh mechanics ideas where you have quantum processes on a on a manifold for your space time. But I think that a similar idea uh can can be used to take your abstract description of a generative model and embed it onto an actual um topological space which is the sort of space-like um organization of a particular brain or a particular machine. Right. Um cool. Yeah. I I think of the saying uh slow is smooth and smooth is fast. Like the speed up sounds like it's going to be the pot of gold at the end of the rainbow, but actually the modularity and the legibility of the software is what is going to get that infrastructure grade long-term templated reuse of cognitive models. And if those take slightly longer to run than some super special case optimized situation, first off, we're only going to get to that optimized runtime from the modular design. And then in in the end I think

it's going to be the compositionality that leads to ecosystems of components that are effective and then speed can be sort of a secondary consideration that's runtime and situation specific. But like the benefits of bringing category theory to active inference, I hope we can draw out over the coming months and years because it really represents a upgrade in how these models are are represented and communicated and they kind of bring an aesthetic possibility with its own grammar that is clearly to be learned. and also a really strong machine readability. So I hope that people who view this are excited to try Disco Py and that we can continue to like look back at the models that have been built and bring them into these notation systems to demonstrate how they can exactly recapitulate and give reproducibility to past examples. And I think that act alone is going to reveal a sort of adjacent possible and it would be awesome if that could all be happening in a flexible environment like Disco Pi. Yeah, I I can't agree more. I mean, I think there's a similar effort that's been going on on the machine learning side where people have taken category theory to formulize existing models and it's always the question of okay, I mean, you can do existing stuff, but what's the um what's the new stuff you can do with it? And I I feel like yeah, active inference is definitely a place where you can look for this these new models where potentially there's um there's something there that you can um describe in a succinct elegant way using categories and show has some experimental value in practice and I guess software is going to be key to turn these theoretical insights into experimental um experimental value and and I think yeah um this This ability of category theory to make bridges between different topics and to to make metaphors become formal is also a good way to uh get inspiration uh from other fields into active inference and and back. So I think um I guess yeah it would be super exciting that the same library gets used by quantum natural language processing people and active inference that have a priority nothing to do with each other. Um, cool. Well, do you have any last comments? And also like what directions are you taking it now with the package or just in your own work? Um, yeah. Yeah, I mean I guess um we're looking very much for contributors especially um people who can bring um projects to apply uh the library in new directions and so um please get in touch on on GitHub or otherwise uh and we'll be yeah very happy to um to welcome new people to the the disco pipe uh contributors and uh in terms of new directions we're pushing it in uh I guess uh yeah the sort of education effort is something we're um trying to invest a lot of time in. So we uh spend a lot of time writing documentation, examples, notebooks. Uh hopefully uh we'll lower the barrier to entry to something that's quite scary at first. The I mean category theory is one of the most intimidating topics out there. So making it more childish and talk about snakes

and spiders and sort of making it fun uh is the part of it. uh and uh I guess yeah in terms of development I think um there's a lot of work on category theory for probabilities um and I think um that has a very strong connection with functional programming like I guess there's this monet um library in high school that uh implements um a lot of um Beijian inference in terms of monads and uh very elegantly in terms of category theory. I think some of that could be ported to to Python um and built into disco pi so that you could um potentially yeah um go beyond sort of toy examples where you have a bunch of categorical distributions and go into something where it's um yeah a very um powerful system that can handle all kinds of uh Beijian inference problems and in the same um same sort of nice looking diagram The future is diagrammed. Yeah, that's I mean matrices were the maths of the 20th century. Um diagrams will be u the maths of the 21st. Cool. Thank you very much. Looking forward to learning more and appreciate all the work and the outreach that you all do. So yeah, thanks a lot for the invite and hope uh get in touch very soon. Thank you. Bye. Take care.